`codesign-mcdp`: A Python Library for Monotone Co-Design Problems

Version 0.1.0 – Reference Manual

May 2026

Abstract

`codesign-mcdp` is a Python library for formulating and solving *Monotone Co-Design Problems* (MCDPs) in the framework of Censi (2015). A design problem is a relation between two posets, a functionality poset F and a resource poset R; given a target functionality, the problem asks for the antichain of minimal resources needed to deliver it. Design problems compose under three operators (series, parallel, feedback), and the resulting class is closed under composition. The library implements the antichain calculus, six primitive design-problem types, the three composition operators, a Kleene fixed-point solver, two high-level builders (an MCDPL-style declarative builder and a modular `System` builder), and a collection of worked examples. This document is the reference manual for version 0.1.0.

Contents

1 Introduction

A *co-design problem* is the joint design of components whose specifications mutually depend on each other. A canonical example: a battery must store enough energy to power an actuator, but the actuator must lift the battery itself, so the battery's mass appears in the energy it has to supply. There is no way to fix one without fixing the other.

Censi (2015) showed that under modest monotonicity assumptions, such problems have a clean algebraic theory: the relations between resources and functionalities form a category closed under series, parallel, and feedback composition, and solutions are computed by a Kleene fixed-point iteration in the lattice of antichains. The original publication included a software tool, `MCDPL`, distributed through `co-design.science`, which exposes the framework via a domain-specific language.

`codesign-mcdp` is a from-scratch Python implementation of the algorithmic core, designed around composable Python objects rather than a separate DSL. It targets users who want to formulate and solve MCDPs from inside their existing Python codebase without context switches to a DSL or external solver. The library is small (about 2000 lines of Python), has no required runtime dependencies beyond the standard library, and is licensed under MIT.

Scope of this document. This manual covers the public API and the mathematical structure it implements. Section 2 recalls the formal background. Sections 4 through 7 describe the data types, primitive DPs, composition operators, and the solver in detail. Sections 11 and 12 document the two high-level builders. Section 13 walks through every example shipped with the library. Section 14 collects modelling guidelines and common pitfalls. Section 15 discusses limitations and future directions.

Conventions. Code listings are typeset in monospaced font. Type signatures use Python's standard notation (`f: F → R`, for example). Mathematical quantities use F, R, h, $\top$, $\bot$, Min, etc.

2 Mathematical Background

This section recalls the formal definitions from Censi (2015) needed to specify the API. Proofs are omitted; the reader is referred to the paper for the full development.

2.1 Posets and Lattices

Definition 2.1 (Poset). *A* poset *(partially ordered set) is a pair $(P, \leq)$ where $\leq$ is a reflexive, antisymmetric, transitive binary relation on the set P.*

Definition 2.2 (Bottom and top). *An element $\bot \in P$ is a* bottom *(least element) if $\bot \leq x$ for every $x \in P$. An element $\top \in P$ is a* top *(greatest element) if $x \leq \top$ for every $x \in P$. A poset has at most one bottom and at most one top.*

Example 2.3 (Reals and Naturals). *The non-negative reals augmented with infinity, $\overline{\mathbb{R}_{\geq 0}} = \mathbb{R}_{\geq 0} \cup \{+\infty\}$, ordered by the usual $\leq$, form a poset with $\bot = 0$ and $\top = +\infty$. Likewise $\overline{\mathbb{N}} = \mathbb{N} \cup \{+\infty\}$.*

Example 2.4 (Product order). *Given posets $(P_1, \leq_1), \ldots, (P_n, \leq_n)$, the product $P_1 \times \cdots \times P_n$ with the componentwise order*

$$(x_1, \ldots, x_n) \leq (y_1, \ldots, y_n) \iff \forall i.\, x_i \leq_i y_i$$

is a poset. The bottom is $(\bot_1, \ldots, \bot_n)$ and the top is $(\top_1, \ldots, \top_n)$.

Definition 2.5 (Antichain). *A subset $A \subseteq P$ is an* antichain *if no two distinct elements of A are comparable: $\forall x, y \in A,\ x \leq y \implies x = y$. We write $\mathsf{A}[P]$ for the set of all antichains of P.*

Definition 2.6 (Min operator). *For any subset $S \subseteq P$, the* Min operator *returns the set of minimal elements:*

$$\mathsf{Min}(S) = \{x \in S : \nexists y \in S.\, y < x\}.$$

$\mathsf{Min}(S)$ is always an antichain. For a finite S, $\mathsf{Min}(S)$ is the Pareto front of S.

2.2 Antichains as a Lattice

The set of antichains of a poset carries a natural order: $A \leq A'$ in $\mathsf{A}[P]$ iff every point in A' is dominated by some point in A. Intuitively, $A \leq A'$ means A is "lower" or "easier", having weaker Pareto-minimal demands.

Proposition 2.7. *$(\mathsf{A}[P], \leq)$ is a complete lattice when P is finite or chain-complete. Joins are computed by union followed by Min. The bottom of $\mathsf{A}[P]$ is $\{\bot_P\}$ (or $\emptyset$ if P has no bottom); the top is $\emptyset$.*

This lattice is the state space of the Kleene iteration (Section 6.3).

2.3 Design Problems

Definition 2.8 (Design Problem). *A* design problem (DP) *is a tuple $\langle F, R, h\rangle$ where F and R are posets and $h : F \to \mathsf{A}[R]$ is a function from functionality requests to antichains of resources. $h(f)$ is interpreted as the Pareto front of minimal resources required to deliver functionality f.*

Definition 2.9 (Monotone Design Problem). *A DP $\langle F, R, h\rangle$ is* monotone *(an MCDP) if h is monotone with respect to the antichain order: $f \leq f'$ in F implies $h(f) \leq h(f')$ in $\mathsf{A}[R]$.*

Intuitively, monotonicity says that demanding more functionality cannot reduce the minimal resources needed. This corresponds to the common-sense engineering principle that you don't get more for less.

2.4 Composition

The class of MCDPs is closed under three operators.

Definition 2.10 (Series). *Given DPs $d_1 = \langle F, M, h_1\rangle$ and $d_2 = \langle M, R, h_2\rangle$ whose interface poset M matches, their* series composition *is*

$$(d_1 \circ d_2)(f) = \mathsf{Min}\left(\bigcup_{r_1 \in h_1(f)} h_2(r_1)\right).$$

Definition 2.11 (Parallel). *Given DPs $d_1 = \langle F_1, R_1, h_1\rangle$ and $d_2 = \langle F_2, R_2, h_2\rangle$, their* parallel composition *is $\langle F_1 \times F_2, R_1 \times R_2, h_1 \otimes h_2\rangle$ where*

$$(h_1 \otimes h_2)(f_1, f_2) = \mathsf{Min}(h_1(f_1) \times h_2(f_2)).$$

Definition 2.12 (Feedback). *Given a DP $d = \langle F \times X, R \times X, h\rangle$ with a common factor X, the* feedback composition *on X is the DP $d^{\dagger X} = \langle F, R, h^{\dagger X}\rangle$ where $h^{\dagger X}(f)$ is the antichain obtained by closing the recursion on X.*

The recursion is solved by Kleene iteration in $\mathsf{A}[R \times X]$, seeded at $\{\bot\}$ and ascending until a fixed point is reached.

2.5 The Kleene Fixed-Point Algorithm

Theorem 2.13 (Censi 2015, Prop. 4)**.** *For an MCDP $d = \langle F \times X, R \times X, h\rangle$ closed on X, define the operator $\Phi_f : \mathsf{A}[R \times X] \to \mathsf{A}[R \times X]$ by*

$$\Phi_f(A) = \mathsf{Min}\left(\bigcup_{r \in A} h(f, r_X) \cap \uparrow r\right),$$

where r_X is the X-component of r and $\uparrow r$ is the upward closure. Then Φ_f is monotone, and the Kleene ascent

$$A_0 = \{\bot\}, \quad A_{k+1} = \Phi_f(A_k)$$

converges to the least fixed point, which equals $h^{\dagger X}(f)$ projected to $\mathsf{A}[R]$.

In implementation, the iteration terminates when one of three conditions holds: $A_{k+1} = A_k$ (fixed point), $A_{k+1} = \emptyset$ (infeasibility), or every point in A_{k+1} has $r_X = \top_X$ (loop axis saturates).

3 Installation and Project Layout

`codesign-mcdp` requires Python 3.9 or newer. The package has no required runtime dependencies; `matplotlib` is needed only for example 05.

Installation. Clone the repository and install in editable mode:

```
git clone https://github.com/your-username/codesign-mcdp.git
cd codesign-mcdp
pip install -e .
```

Optional extras.

- `pip install -e ".[viz]"` adds matplotlib for the visualisation example.
- `pip install -e ".[nb]"` adds the Jupyter tooling needed to (re)generate the notebooks.

Repository layout.

```
codesign-mcdp/
  codesign/           the Python package (9 modules)
  examples/           eight runnable example scripts
  notebooks/          the same examples as executed .ipynb notebooks
  tests/              smoke tests
  docs/               this manual plus rendered trace images
  build_notebooks.py  script to regenerate the notebooks
  pyproject.toml      packaging metadata, PEP 621
  .github/workflows/  CI configuration
```

4 Core Data Types

This section documents the data types provided by `codesign.posets` and `codesign.antichains`.

4.1 Posets

The `Poset` abstract base class has the following interface:

```
class Poset(ABC):
    name: str

    def leq(self, x, y) → bool: ...       # is x ≤ y?
    def eq(self, x, y) → bool: ...
    def lt(self, x, y) → bool: ...
    def bottom(self): ...                  # least element, if any
    def top(self): ...                     # greatest element, if any
    def is_bottom(self, x) → bool: ...
    def is_top(self, x) → bool: ...
    def join(self, x, y): ...              # least upper bound
    def format(self, x) → str: ...
```

Four concrete poset classes are provided.

`Reals(unit="")`. The non-negative reals augmented with $+\infty$. Bottom is 0.0; top is `math.inf`. The optional `unit` string is used for pretty-printing only.

`Naturals(unit="")`. The non-negative integers augmented with $+\infty$. Bottom is 0; top is `math.inf`.

`Ports(components: dict[str, Poset])`. The componentwise-ordered product of named factor posets. Elements are Python dicts mapping component names to values. The bottom is the dict of all bottoms; the top is the dict of all tops. `Ports` can be nested freely: a component poset may itself be a `Ports`. This is the type used for every design problem's F and R.

The name reflects the library's vocabulary: each named component is a *port* of the design problem, addressable by name in the constraint DSL. For backward compatibility the older name `NamedProduct` remains exported as an alias, so existing code that imports `NamedProduct` continues to work; new code should prefer `Ports`.

`Discrete(elements, leq=None)`. A finite poset with explicit element set and a caller-supplied `leq` predicate. Useful for catalog-like enumerations where the order is not the default discrete order.

4.2 Antichains

The `Antichain` class wraps a set of poset elements together with the poset they live in. The constructor normalises the input by removing dominated points and deduplicating.

```
class Antichain:
    poset: Poset
    points: tuple

    @classmethod
    def of_bottom(cls, poset) → "Antichain": ...
    @classmethod
    def empty(cls, poset) → "Antichain": ...
    @classmethod
    def singleton(cls, poset, point) → "Antichain": ...
    @classmethod
    def from_set(cls, poset, points) → "Antichain": ...

    def union_min(self, other) → "Antichain": ...
    def filter_above(self, floor) → "Antichain": ...
```

```python
    def leq(self, other) → bool: ...
    def eq(self, other) → bool: ...
    def has_any_top(self) → bool: ...
    def is_empty(self) → bool: ...
```

The two key operations on antichains are `union_min`, which takes the union of two antichains and re-applies Min, and `filter_above`, which keeps only points dominating a given floor. Together they implement the body of the Kleene iteration in Theorem 2.13.

5 Primitive Design Problem Types

Six concrete subclasses of `DesignProblem` are provided, each appropriate for a different style of relation between functionality and resources.

5.1 `AlgebraicDP`

For relations where each resource is a closed-form monotone function of the functionality.

```python
AlgebraicDP(
    F: Ports,
    R: Ports,
    equations: dict[str, Callable[[dict], float]],
    name: str = "algebraic",
)
```

The `equations` dictionary maps each resource name to a callable taking a functionality dict and returning a scalar. `h(f)` returns a singleton antichain.

Example 5.1. *A battery sized by specific energy:*

```python
battery = AlgebraicDP(
    F=Ports({"capacity": Reals(unit="J")}),
    R=Ports({"mass": Reals(unit="kg")}),
    equations={"mass": lambda f: f["capacity"] / 1.8e6},
)
```

5.2 `FunctionDP`

For relations where the user supplies `h` directly as a function from functionality dicts to antichains. Use this when the relation is multi-valued (a true Pareto front) or has branching logic.

```python
FunctionDP(
    F: Ports,
    R: Ports,
    h_fn: Callable[[dict], Antichain],
    name: str = "function",
)
```

Example 5.2 (A two-option amplifier)**.** *The same audio amplifier output spec admits two implementations: a class-A topology (clean, hot) or a class-D topology (efficient, harsh).* `h(f)` *emits both points, so the engineer sees the real tradeoff:*

```python
def h_amp(f):
    watts = f["watts"]
    return Antichain.from_set(R, [
        {"power_in": watts / 0.25, "thd": 0.001},   # class-A
        {"power_in": watts / 0.90, "thd": 0.020},   # class-D
    ])

```

```
amp = FunctionDP(
    F=Ports({"watts":    Reals(unit="W")}),
    R=Ports({"power_in": Reals(unit="W"),
                "thd":      Reals()}),
    h_fn=h_amp,
)
```

5.3 CatalogDP

For selecting from a finite catalog of implementations (motors, batteries, sensors).

```
CatalogDP(
    F: Ports,
    R: Ports,
    catalog: list[CatalogEntry],
    name: str = "catalog",
)

CatalogEntry(
    name: str,
    provides: dict[str, Any],
    costs: dict[str, Any],
)
```

For a request f, $h(f)$ is the antichain of `entry.costs` for every entry whose `provides` dominates f in F. Entries that are dominated by another feasible entry are pruned by Min in R.

Example 5.3 (A small motor catalog). *Three motors with incomparable mass–cost tradeoffs at the same torque rating.* h *returns whichever entries cover the requested torque, then* Min *drops dominated ones automatically:*

```
motor = CatalogDP(
    F=Ports({"torque": Reals(unit="N*m")}),
    R=Ports({"mass":   Reals(unit="kg"),
                "cost":   Reals(unit="USD")}),
    catalog=[
        CatalogEntry(name="Tiny",   provides={"torque": 2.0},
                        costs={"mass": 0.2, "cost": 30.0}),
        CatalogEntry(name="Light",  provides={"torque": 8.0},
                        costs={"mass": 0.5, "cost": 200.0}),
        CatalogEntry(name="Heavy",  provides={"torque": 8.0},
                        costs={"mass": 0.8, "cost": 120.0}),
    ],
)
# motor.h({"torque": 7.0}) yields the antichain
# Antichain[(mass=0.5, cost=200), (mass=0.8, cost=120)]
```

The Tiny entry is filtered out (does not cover 7 N·m). Light and Heavy are mutually incomparable, so both survive the Min.

5.4 ConstraintDP

A feasibility predicate plus a scalar cost, lifted to Min. Useful when the resource-functionality relation is most natural to express as *is this choice good enough, and how much does it cost?*.

```
ConstraintDP(
    F: Ports,
    R: Ports,
    sampler: Callable[[dict], Iterable[dict]],
    feasible: Callable[[dict, dict], bool],
```

```
    cost: Callable[[dict, dict], dict],
    name: str = "constraint",
)
```

The `sampler` yields candidate resource dicts for the given functionality; `feasible` filters them; `cost` maps the feasible ones into the resource poset, and the result is reduced by Min.

5.5 `ODE_DP`

Derives a monotone resource relation from a differential equation $\dot{x} = \text{rhs}(x, t, f)$. Two modes are supported.

Final value integrates the ODE from $t = 0$ to $t = t_{\text{end}}$ by explicit Euler and reports the extracted resources of the final state.

Steady state solves $\text{rhs}(x, \cdot, f) = 0$ for x by Newton iteration and reports the extracted resources of the steady-state value.

```
ODE_DP(
    F: Ports,
    R: Ports,
    rhs: Callable,
    extract: Callable[[state], dict],
    mode: str,              # "final_value" or "steady_state"
    t_end: float = 1.0,
    n_steps: int = 100,
    x0_fn: Callable[[dict], Any] = ...,
    name: str = "ode",
)
```

Example 5.4 (A heater at thermal steady state). *A wall heater whose steady-state input power is recovered from Newton's law of cooling, $\dot{T} = (P - k(T - T_\infty))/C$. The heater's resource is the electrical power required to hold the target temperature:*

```
def rhs(state, t, f):
    T = state["T"]
    k, T_inf, C = 0.5, 20.0, 1000.0
    return {"T": (f["power_in"] - k * (T - T_inf)) / C}

heater = ODE_DP(
    F=Ports({"target_T":  Reals(unit="C"),
                         "power_in":  Reals(unit="W")}),
    R=Ports({"steady_T":  Reals(unit="C")}),
    rhs=rhs,
    extract=lambda state: {"steady_T": state["T"]},
    mode="steady_state",
    x0_fn=lambda f: {"T": 20.0},
)
```

At steady state Newton iteration finds $T = T_\infty + P/k$, which is recovered exactly by the solver.

5.6 `UncertainDP`

Brackets an unknown h with a known lower and upper bound. The optimistic mode (`lower`) yields a lower bound on the minimal resources; the pessimistic mode (`upper`) yields an upper bound. Section VII of Censi (2015) develops the theory.

```
UncertainDP(
    F: Ports,
```

```
    R: Ports,
    lower: DesignProblem,
    upper: DesignProblem,
    mode: str = "upper",
    name: str = "uncertain",
)
```

`with_mode("lower" | "upper")` returns the bracket selected for solving.

Example 5.5 (A battery with uncertain specific energy). *Specific energy of the cell chemistry is known only to lie in* $[1.6, 2.0]$ *MJ/kg. The optimistic and pessimistic brackets give lower and upper bounds on the required mass:*

```
F = Ports({"capacity": Reals(unit="J")})
R = Ports({"mass":     Reals(unit="kg")})

lo = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 2.0e6})
hi = AlgebraicDP(F, R, {"mass": lambda f: f["capacity"] / 1.6e6})

battery = UncertainDP(F, R, lower=lo, upper=hi)

# Solve both brackets and report the interval.
m_opt  = solve(battery.with_mode("lower"), {"capacity": 3.6e6})
m_pess = solve(battery.with_mode("upper"), {"capacity": 3.6e6})
# m_opt  → Antichain[(mass=1.80 kg)]
# m_pess → Antichain[(mass=2.25 kg)]
```

6 Composition Operators

6.1 Series

```
series(dp1: DesignProblem, dp2: DesignProblem) → Series
Series(dp1, dp2)
```

Requires `dp1.R == dp2.F` structurally (same component names and posets). Implements the series definition: for each point of $h_1(f)$ run h_2 and take the union Min.

Example 6.1 (Battery feeding an actuator). *A battery converts energy demand into mass; the actuator converts that mass demand into a power draw. Series composition feeds the battery's output directly into the actuator's input:*

```
battery = AlgebraicDP(
    F=Ports({"capacity": Reals(unit="J")}),
    R=Ports({"mass":     Reals(unit="kg")}),
    equations={"mass": lambda f: f["capacity"] / 1.8e6},
)
actuator = AlgebraicDP(
    F=Ports({"mass":     Reals(unit="kg")}),
    R=Ports({"power":    Reals(unit="W")}),
    equations={"power": lambda f: 10.0 * f["mass"] ** 2},
)
chain = series(battery, actuator)
# solve(chain, {"capacity": 3.6e6}) maps energy → power directly.
```

6.2 Parallel

```
par(dp1: DesignProblem, dp2: DesignProblem) → Parallel
Parallel(dp1, dp2)
```

Requires non-overlapping component names in F_1, F_2 and in R_1, R_2. The output F and R are the concatenated `Ports`s. The antichain is the Cartesian product followed by Min.

Example 6.2 (Two independent subsystems). *A drone has a propulsion subsystem (energy in → mass out) and a sensing subsystem (sample rate in → power out). They share no ports and run in parallel; the composite DP takes both inputs and emits both outputs:*

```
propulsion = AlgebraicDP(
    F=Ports({"energy":      Reals(unit="J")}),
    R=Ports({"prop_mass":   Reals(unit="kg")}),
    equations={"prop_mass": lambda f: f["energy"] / 1.8e6},
)
sensor = AlgebraicDP(
    F=Ports({"sample_rate": Reals(unit="Hz")}),
    R=Ports({"sensor_pwr":  Reals(unit="W")}),
    equations={"sensor_pwr": lambda f: 0.05 * f["sample_rate"]},
)
combined = par(propulsion, sensor)
# combined.F has both energy and sample_rate;
# combined.R has both prop_mass and sensor_pwr.
```

6.3 Loop

```
loop(inner: DesignProblem, axis: str) → Loop
Loop(inner, axis="...")
```

Closes the feedback loop on a single named axis that must appear in both `inner.F` and `inner.R`. The resulting DP has `F = inner.F` minus the axis and `R = inner.R` minus the axis. Calling `h(f)` triggers the Kleene fixed-point iteration described in Theorem 2.13.

Example 6.3 (Battery mass feedback). *A drone's battery must carry both the payload and its own mass. Closing the loop on* `battery_mass` *converts the self-referential inequality into a fixed-point problem that the Kleene solver handles automatically:*

```
inner = AlgebraicDP(
    F=Ports({"payload":       Reals(unit="kg"),
                     "battery_mass":  Reals(unit="kg")}),
    R=Ports({"battery_mass":  Reals(unit="kg")}),
    equations={
        "battery_mass":
            lambda f: 0.5 * (f["payload"] + f["battery_mass"]),
    },
)
drone = loop(inner, axis="battery_mass")
# drone.F = {"payload"}, drone.R = {}
# solve(drone, {"payload": 1.0}) iterates to the fixed point.
```

The closed-form solution is $m^\star = p/(1 - 0.5) = 2p$, which the Kleene iteration recovers in a handful of steps.

Remark 6.4 (Multiple feedback loops). *`Loop` closes one axis at a time. To close several loops, chain them: `loop(loop(inner, axis="x"), axis="y")`. The `System` builder (Section 12) closes all feedback loops simultaneously over a single bundled axis, which is usually more convenient.*

7 The Solver

7.1 Top-level entry point

```
solve(
    dp: DesignProblem,
    functionality: dict | None,
    *,
    max_iter: int = 200,
    record_trace: bool = False,
) → SolveResult
```

The result is a dataclass with the following fields:

```
@dataclass
class SolveResult:
    antichain: Antichain      # the Pareto front (in dp.R)
    iterations: int           # Kleene steps taken (0 if no loop)
    converged: bool           # True if a fixed point was reached
    feasible: bool            # True if antichain has finite, nonempty points
    trace: list[Antichain]    # the sequence A_0, A_1, ... if recorded
```

Example 7.1 (Inspecting a solve result)**.** *The full lifecycle of a solve: build the DP, call* `solve`*, introspect the result, and pick a single design with* `minimize_cost`*:*

```
result = solve(drone, {"endurance":      300.0,
                       "extra_payload":  0.5,
                       "extra_power":    5.0},
               max_iter=200, trace=True)

print(result.status)          # "converged"
print(result.iterations)      # e.g. 17
print(result.feasible)        # True
print(result.antichain)       # Antichain[(total_mass=0.56 kg)]

# Scalarise: cheapest design under a custom cost function
best = minimize_cost(result, cost_fn=lambda r: r["total_mass"])
print(best)                   # {"total_mass": 0.5602}
```

7.2 The Kleene iteration

```
kleene_loop(
    loop_dp: Loop,
    f_outer: dict,
    *,
    max_iter: int = 200,
    record_trace: bool = False,
    trace_out: list | None = None,
    info_out: dict | None = None,
) → Antichain
```

The body of the iteration is direct from Theorem 2.13. The implementation adds:

- a *divergence cap* of 10^{30} that converts an iterate exceeding the cap to $\top$, so floating-point overflow becomes infeasibility rather than `inf`/`nan`;
- `try`/`except` around the inner h to catch `OverflowError`, `ValueError`, and `ZeroDivisionError`, again mapping to $\top$;
- three termination conditions: fixed point reached, antichain empty (infeasible), or every point's loop axis equal to $\top$ (provably infeasible).

7.3 Scalar minimisation over an antichain

```
minimize_cost(
    result: SolveResult,
    cost_fn: Callable[[dict], float],
) → dict | None
```

Returns the single resource bundle in `result.antichain` that minimises `cost_fn`, or `None` if the result is infeasible. This is the scalarisation step that turns a Pareto front into a single engineering choice.

7.4 Observability: trace, verbose, callback, status

The solver can be made to report its progress in three independent ways, and its termination reason is exposed through a structured `status` field that is orthogonal to `feasible`.

The status field. `SolveResult.status` is one of three strings:

`"converged"` The iteration reached a fixed point (or provably hit $\top$) within `max_iter` steps. The answer in `result.antichain` is the algorithm's verdict; `feasible` tells you whether that verdict is finite (a real design) or $\top$ (no design exists).

`"max_iter"` The iteration was cut off at `max_iter`. The current antichain may or may not be near a fixed point. Usually a sign to increase `max_iter`, or to look for an unintentional non-monotonicity in the model.

`"diverged"` At least one numeric component crossed the divergence cap of 10^{30} before the iteration could settle, and the solver stopped. Distinguishes pure numerical blow-up from clean $\top$-infeasibility.

The previous Boolean `converged` field is preserved as a backward-compatibility alias for `status == "converged"`.

Structured trace. Passing `trace=True` to `solve` populates `result.trace` with a list of `TraceEntry` dataclasses, one per Kleene step (plus the seed at iteration 0):

```
@dataclass
class TraceEntry:
    iteration: int            # 0 = seed, then 1, 2, ...
    antichain: Antichain      # snapshot at this step
    n_points:  int            # convenience: len(antichain)
    delta:     float | None   # max absolute change (numeric)
                              # or 0/1 (discrete); None at iter 0
    elapsed_ms: float         # wall time for this step alone
```

With `trace=False` (the default) the field is `None`, so the cost of unused tracing is zero.

Verbose printing. The integer `verbose` controls live output:

`verbose=0` (default) silent.

`verbose=1` one summary line at the end, e.g. `[solve] converged:  17 iters, |A|=1, total=2.0ms, feasible=True`.

`verbose=2` one line per iteration, showing the delta, the antichain size, and the step's wall time. Useful for watching oscillating fixed points.

Iteration callback. Passing `on_iteration=callable` registers a callback that receives each `TraceEntry` as soon as it is produced. Useful for live plots, custom logging, or interrupting an iteration that looks pathological:

```
def log_every_5(entry):
    if entry.iteration % 5 == 0:
        print(f"iter {entry.iteration}: delta={entry.delta:.3e}")

solve(dp, f, on_iteration=log_every_5)
```

Example 7.2 (Warm-starting a parameter sweep). *When sweeping a parameter (load, mission length, demand level), the fixed point at one parameter is a great seed for the next. Pass the previous `SolveResult` (or its `antichain`) via `start_from=` and the Kleene iteration picks up from there instead of restarting at* $\perp$*:*

```
prev = None
results = []
for L in np.linspace(5.0, 30.0, 50):
    r = solve(dp, {"daily_load_kwh": float(L), ...},
              max_iter=400, start_from=prev)
    results.append(r)
    prev = r            # reuse the converged inner antichain
```

For the microgrid example (notebook 13) this reduces the cumulative iteration count by roughly 10 % across the sweep, more for finer parameter grids.

Example 7.3 (Plotting convergence after a trace). *With `trace=True`, the result carries the entire iteration history; a single `viz.plot_convergence` call renders the delta-vs-iteration semilog:*

```
from codesign import viz

r = solve(drone, f, trace=True, max_iter=200)
ax = viz.plot_convergence(r)
ax.set_title(f"converged in {r.iterations} steps")
```

8 Uncertainty

Two kinds of parameter uncertainty are supported, declared on `Module` instances and consumed by the same `solve` entry point. Both are independent of the model's structure: a module can have set-based uncertainty, stochastic uncertainty, both, or neither.

8.1 Declaration

A `Module` carries optional attributes:

```
class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}

    def __init__(self, specific_energy=1.8e6, efficiency=0.85):
        self.specific_energy = specific_energy
        self.efficiency = efficiency
        super().__init__()

    def h(self, f):
        return {"mass": f["capacity"]
                        / (self.specific_energy * self.efficiency)}

b = Battery()
```

```
b.uncertain_set  = Box(...)          # deterministic, set-based
b.uncertain_dist = Stochastic(...)   # statistical
```

The uncertainty solver discovers every such module by walking the `_codesign_modules` attribute that `System.build` attaches to the returned DP. Before each solve, the module's named parameters are reassigned; afterwards, their nominal values are restored. The user's original `Battery()` instance is unchanged.

8.2 Set-based uncertainty (deterministic)

`Box`. Axis-aligned interval product. Each parameter is declared as `(lo, hi)` or `(lo, hi, direction)`, where `direction` is one of the four tokens `"more_is_better"`, `"more_is_worse"`, `"less_is_better"`, `"less_is_worse"`. With all directions declared, the worst case is a single corner read off in constant time:

```
b.uncertain_set = Box(
    specific_energy=(1.6e6, 2.0e6, "more_is_better"),
    efficiency=(0.80, 0.90, "more_is_better"),
)
# worst case = (specific_energy=1.6e6, efficiency=0.80)
```

When some directions are undeclared, all 2^n endpoint combinations are evaluated and the worst is kept; cheap when n is modest.

`Ellipsoid`. An n-D ellipsoid $(p-c)^\top \Sigma^{-1}(p-c) \le 1$ in parameter space. The covariance Σ encodes both scale and correlation between parameters, so an ellipsoid is a more honest model than a box when parameters are believed to vary together:

```
b.uncertain_set = Ellipsoid(
    center={"specific_energy": 1.8e6, "efficiency": 0.85},
    cov=[
        [1.0e10, -2.0e3],
        [-2.0e3,  2.5e-3],
    ],
    params=["specific_energy", "efficiency"],
    directions={
        "specific_energy": "more_is_better",
        "efficiency":      "more_is_better",
    },
)
```

With all directions declared, the worst case is the boundary point $c + L\,u^\star$, where L is the Cholesky factor of Σ and $u^\star$ is the unit vector in the direction of badness. Otherwise, the boundary is sampled (controlled by `boundary_samples`) and the worst sample is returned.

2D conveniences. `Disk(center, radius, ...)` and `Circle(center, radius, ...)` reduce to the isotropic ellipsoid $\Sigma = r^2 I$. For monotone systems with declared directions, the two are equivalent: the worst case lies on the boundary either way.

8.3 Stochastic uncertainty

`Stochastic`. A joint distribution built from named scipy-stats frozen marginals plus a copula:

```
from scipy import stats

b.uncertain_dist = Stochastic(
    marginals={
        "specific_energy": stats.uniform(loc=1.6e6, scale=0.4e6),
        "efficiency":      stats.uniform(loc=0.80, scale=0.10),
```

```
    },
    copula=GaussianCopula(correlation=[[1.0, 0.4],
                                       [0.4, 1.0]]),
)
```

Sampling is by the standard copula–marginal trick: the copula generates correlated uniforms in $[0,1]^d$; the marginal `ppf`s map them to the named parameter axes.

Copulas. `Independence()` is the default (coordinatewise uniform). `GaussianCopula(correlation)` uses the Gaussian copula with the given correlation matrix; samples are drawn by Cholesky factorisation followed by Φ on each coordinate. Subclassing `Copula` requires only one method, `sample_uniform(n, d, rng)`, so other copulas (e.g. Clayton, Gumbel) drop in without further plumbing.

8.4 Querying

The same `solve` entry point accepts an `uncertainty` list and returns a richer object:

```
result = solve(
    dp, f,
    uncertainty=["worst_case", "mean", "p95", "cvar95", "samples"],
    n_samples=1000,
    rng_seed=42,
)

result.worst_case        # SolveResult at the worst point of the set
result.mean              # dict[r_port → mean across MC samples]
result.p95               # dict[r_port → 95th percentile]
result.cvar95            # dict[r_port → CVaR at 95% level]
result.samples           # list[Antichain], one per MC sample
result.feasibility_rate  # fraction of feasible MC samples
```

The allowed labels are:

`"worst_case"` The deterministic worst over every module's `uncertain_set`. One canonical solve at the worst-case parameter point.

`"mean"`, `"p95"`, `"cvar95"` Marginal statistics over Monte Carlo samples, drawn from every module's `uncertain_dist`. The CVaR (conditional value at risk) at the 95% level is the mean of the worst 5% of samples.

`"samples"` The raw antichain per MC sample, for the user to aggregate however they like.

For a single solve call you typically observe the ordering

$$\text{nominal} < \text{mean} < \text{p95} < \text{CVaR95} < \text{worst\_case},$$

which separates the four standard ways of reporting an answer "under uncertainty." Notebooks 11 and 12 visualise this for the example-7 drone.

Example 8.1 (Box worst-case of a drone battery)**.** *Specific energy and efficiency of the battery cell live in declared ranges, each with a direction of badness. The worst-case mass is the corner where both parameters take their worst endpoint:*

```
bat = Battery()  # the Module subclass from example~7
bat.uncertain_set = Box(
    specific_energy=(1.6e6, 2.0e6, "more_is_better"),
    efficiency=(0.80, 0.90, "more_is_better"),
)
# ... wire ‘bat‘ into the drone system ...
```

```python

result = solve(drone, f, uncertainty=["worst_case"])
worst_mass = list(result.worst_case.antichain.points)[0]["total_mass"]
# worst_mass = 0.5760 kg, versus 0.5602 kg nominal
```

Example 8.2 (Monte Carlo with a Gaussian copula). *The same battery, now with stochastic specific energy and efficiency, correlated at $\rho = 0.4$. One* `solve` *call returns all four summary statistics:*

```python
from scipy import stats

bat.uncertain_dist = Stochastic(
    marginals={
        "specific_energy": stats.uniform(loc=1.6e6, scale=0.4e6),
        "efficiency":      stats.uniform(loc=0.80, scale=0.10),
    },
    copula=GaussianCopula(correlation=[[1.0, 0.4],
                                       [0.4, 1.0]]),
)

res = solve(drone, f,
            uncertainty=["mean", "p95", "cvar95", "samples"],
            n_samples=1000, rng_seed=42)
# res.mean["total_mass"]    → 0.5614 kg
# res.p95["total_mass"]     → 0.5729 kg
# res.cvar95["total_mass"]  → 0.5742 kg
# len(res.samples)          → 1000
```

9 Online Learning

When a co-design problem has many discrete candidates (catalogue entries, robot types, component families) and each candidate's inner solve is non-trivial, the naive "evaluate every entry" strategy is wasteful. The `codesign.online` module implements the compositional online learner of Alharbi, Dahleh & Zardini (`arXiv:2604.22624`): maintain history-dependent bounds on each candidate's inner-solve output, evaluate the most promising one under an upper-confidence rule, then prune any candidate whose lower bound is already dominated by the incumbent.

9.1 Optimistic evaluators

The bound machinery is encapsulated in a small class hierarchy. `OptimisticEvaluator` is the abstract base; subclasses implement `bound(candidate)` returning a pair of dicts `(lower, upper)` mapping each numeric R component to its current lower and upper bound at the queried feature point. Tighter bounds prune more candidates; looser bounds are always safe.

Three concrete evaluators are provided:

Monotonicity. `MonotonicityEvaluator(features, r_components)` assumes the inner-solve output is component-wise monotone in the named features. Given an observation at feature point f_0 with antichain-min R_0, any candidate whose features dominate f_0 has resources $\geq R_0$ (lower bound); any candidate dominated by f_0 has resources $\leq R_0$ (upper bound). Aggressive when applicable. Only correct if the monotonicity assumption genuinely holds: choosing a derived feature (e.g. `cost_per_capacity = unit_cost / (speed * payload)`) is often what makes this safe in practice.

Lipschitz. `LipschitzEvaluator(features, r_components, L)` assumes $|h(c_1) - h(c_2)| \leq L \cdot \|features(c_1) - features(c_2)\|$ (Euclidean distance, per R component). Each observation tightens the bound by a cone of slope L. The constant can be a scalar (same for every R component) or a dict per component. Safe default: with a sensible L the bound never prunes a Pareto-optimal candidate.

Linear-parametric. `LinearParametricEvaluator(features, r_components, confidence=2.0, min_obs=3)` fits a running OLS model $h_k(c) \approx a_k + b_k \cdot features(c)$ and bounds new queries by a confidence band of width `confidence` times the residual standard deviation. Fastest in practice. Risk: when the true relationship is nonlinear, the confidence band can be too tight and a Pareto-optimal candidate can be wrongly eliminated.

Gaussian process. `GaussianProcessEvaluator(features, r_components, length_scale=0.3, sigma_f=1.0, noise=1e-3, confidence=2.0, min_obs=3)` fits a zero-mean GP with an RBF kernel and bounds new queries by a confidence band on the predictive standard deviation. Implemented in pure numpy. The right tool when the response surface has strong feature interactions or local nonlinearity that a linear regression misses. For nearly-additive surfaces the linear-parametric evaluator is empirically competitive and significantly faster.

Subclassing. Custom evaluators only need to override `bound(candidate)`; the base class handles observation recording and the default fallback bound of $(0, +\infty)$.

9.2 The online solver

```
result = solve_online(
    candidate_fn,          # candidate_fn(candidate) → DP
    functionality,         # outer F vector
    *,
    candidates,            # list of feature dicts
    evaluator,             # OptimisticEvaluator instance
    budget=None,           # max inner solves; None = unbounded
    max_iter=200,          # forwarded to each inner solve
    verbose=0,
    warm_start=None,       # seed indices or n for farthest-point
    picker="lcb",          # "lcb", "ucb", "random", or callable
)
```

Algorithm (a simplified Algorithm 2 from Alharbi et al.):

1. Bound every remaining candidate via the evaluator.
2. Eliminate every candidate whose lower bound is already dominated by the current incumbent antichain.
3. Pick the most promising survivor by UCB on its lower-bound sum.
4. Run the inner solve at that candidate; observe the result in the evaluator; merge into the incumbent via `union_min`.
5. Repeat until the candidates are exhausted or the budget is hit.

The returned `OnlineResult` exposes:

`antichain` Min over the evaluated, surviving candidates.

`n_evaluated`, `n_eliminated`, `n_candidates` Bookkeeping for the elimination cascade.

`evaluated_ids`, `eliminated_ids`, `incumbent_ids` Lists of indices into the original candidate list.

`history` Per-iteration log: `{pick, antichain, remaining, evaluated, eliminated}`.

Example 9.1 (Fleet sizing across 200 robot types)**.** *A logistics service has a 200-entry catalogue of candidate robot types, each described by four features. The Lipschitz evaluator caps how fast the inner-solve output can change with the features, so a single observation tightens the bound on every other candidate. With the right L the answer is exact:*

```
def candidate_fn(robot):
    capacity = robot["speed"] * robot["payload"]
    return AlgebraicDP(F, R, {
        "total_cost":   lambda f, cap=capacity, uc=robot["unit_cost"]:
                        (f["target_throughput"] / cap) * uc,
        "total_energy": lambda f, ek=robot["energy_per_km"]:
                        f["target_range"] * ek * 24.0,
    })

ev = LipschitzEvaluator(
    features=["speed", "payload", "unit_cost", "energy_per_km"],
    r_components=["total_cost", "total_energy"],
    L={"total_cost": 300.0, "total_energy": 30.0},
)

result = solve_online(
    candidate_fn, mission,
    candidates=robot_catalog,        # list of 200 feature dicts
    evaluator=ev,
)
# result.n_evaluated   = 192 / 200
# result.n_eliminated  = 8
# result.antichain     = 5 Pareto-optimal robot types
```

Swapping in `MonotonicityEvaluator(features=["cost_per_capacity", "energy_per_km"], ...)` *cuts the evaluation count to about 13 by exploiting the structural monotonicity of the derived feature.*

9.3 Warm-start, picker strategies, and the GP evaluator

The basic online solver has three small extensions that significantly expand the tuning space without complicating the core algorithm.

Warm-start. A new `warm_start` argument to `solve_online` pre-populates the evaluator with seed observations before the picker takes over. It accepts either a list of candidate indices (manually picked corner runs) or an integer n that triggers a greedy farthest-point heuristic over the feature space. The motivation is concrete: the `MonotonicityEvaluator`'s lower bounds only tighten for candidates whose features dominate every observation in the partial order. Without observations at the low-feature corner where the Pareto front typically lives, the picker explores indiscriminately and the evaluator remains uninformative throughout the budget. In example 16, evaluating the example with four corner warm-start runs lifts Monotonicity's Pareto recovery from zero to one of four classes; a hybrid with one of the other evaluators is the natural next step for cases where this is not enough.

Pluggable picker strategies. A new `picker` argument selects the candidate-scoring rule. Built-in options: `"lcb"` (the previous default, minimises the sum of lower-bound components, pure exploitation of the optimistic estimate); `"ucb"` (lower bound minus a $\kappa\cdot$ (upper $-$ lower) exploration bonus, tunable via the tuple form `("ucb", {"kappa": 1.0})`); and `"random"` (uniform baseline for comparing the value of structural priors). Custom callables are accepted and receive `(lower, upper, r_components, **kwargs)`. The strategies that matter in practice are LCB for confident priors and UCB with $\kappa \approx 0.3$ to 0.5 when the prior is uncertain enough that pure exploitation gets stuck in a local region.

Gaussian-process evaluator. A new `GaussianProcessEvaluator` class uses a zero-mean GP with an RBF kernel, implemented in pure numpy. Hyperparameters are `length_scale` (default 0.3, suitable for features in $[0, 1]$), `sigma_f` (default 1.0, rescaled per-output by the empirical observation standard deviation), `noise` (default 10^{-3} jitter), `confidence` (default 2.0σ for roughly 95% coverage), and `min_obs` (default 3). The GP is more expressive than the linear-parametric evaluator and captures local nonlinearity that a global linear fit misses. For nearly-additive response surfaces such as the example 16 bioprocess effect model the two are empirically close, with `LinearParametricEvaluator` preferred for its simplicity; GP is the right tool when the response has strong feature interactions or local peaks that linear regression cannot represent.

10 Visualisation

The `codesign.viz` module exposes four helpers built on `matplotlib` and a small GraphViz emitter:

`plot_antichain(result, axes)` Scatter the antichain on the chosen 2D or 3D axes. Accepts a `SolveResult`, an `UncertaintyResult` (uses its worst case), or a bare `Antichain`. The 2D version optionally shades each point's dominated region for visual clarity.

`plot_convergence(result)` Semilog plot of the Kleene delta against iteration index. Accepts a `SolveResult` with a trace or a trace list directly. Useful for diagnosing oscillating fixed points and tuning `max_iter`.

`plot_uncertainty(unc_result, port, nominal=None)` Histogram of the Monte Carlo samples for the named R port, with the nominal, mean, p95, and CVaR95 marked as vertical lines.

`to_dot(dp, name="dp")` GraphViz dot string for the system structure: modules as boxes, outer ports as labelled rectangles, constraint connections as edges. Render with `dot -Tpng` or paste into an online viewer.

All four accept an optional `ax=...` argument so they compose into larger figures. The module is imported as `from codesign import viz`.

Example 10.1 (Pareto front of a vehicle co-design)**.** *The vehicle from worked example 8 has two Pareto-incomparable designs at the Small-parcel mission. A single call paints them on a mass–cost plane and shades the dominated quadrants so the front is visible at a glance:*

```python
from codesign import viz
import matplotlib.pyplot as plt

result = solve(vehicle, {"payload": 2.0, "mission_energy": 2.0e5})
viz.plot_antichain(result, axes=["total_mass", "total_cost"])
plt.show()
```

For three resources, pass three axis names to get a 3D scatter.

Example 10.2 (Uncertainty histogram with summaries). *Visualising the Monte Carlo output of a stochastic solve: the histogram of total mass across MC samples, with the nominal, mean, p95, and CVaR95 each drawn as a vertical line:*

```
res = solve(drone, f,
            uncertainty=["mean", "p95", "cvar95", "samples"],
            n_samples=1000, rng_seed=42)
ax = viz.plot_uncertainty(res, port="total_mass",
                          nominal=0.5602, bins=30)
```

The function reads the requested summaries off the result and labels them automatically.

Example 10.3 (System graph in GraphViz dot). *For documentation or for understanding someone else's model, the constraint graph of a built `System` can be dumped as dot:*

```
dot = viz.to_dot(microgrid, name="microgrid")
# Render externally:
# echo "<dot output>" | dot -Tpng > microgrid.png
```

Modules become boxes, outer F ports become source rectangles, outer R ports become sink rectangles, and constraint connections become directed edges labelled with the linking expression.

11 The MCDP Builder

The `MCDP` class in `codesign.mcdpl` provides an MCDPL-style declarative syntax for a single self-contained design problem.

```
with MCDP("name") as m:
    m.provides("functionality_name", unit="...")    # outer F port
    m.requires("resource_name",     unit="...")    # outer R port

    m.constraint("resource_name", lambda f: <expr>)   # closed-form
    # or
    m.rule(lambda f: Antichain.from_set(...))         # multi-valued

    m.loop_on("axis_name")    # optional, may be repeated

dp = m.build()
```

The builder emits an `AlgebraicDP` or `FunctionDP` internally, then wraps it in `Loops` as requested by `loop_on`. Multiple `constraint` calls on the same target are joined (max). Mostly useful when the model has a small number of variables and reads naturally as a list of $\geq$ inequalities, mirroring the paper's `mcdp { ... }` blocks.

Example 11.1 (Drone as a single MCDP block). *The Fig. 48 drone rewritten as a flat MCDP, with one closed loop on `battery_mass`:*

```
with MCDP("drone") as m:
    m.provides("endurance",     unit="s")
    m.provides("extra_payload", unit="kg")
    m.provides("extra_power",   unit="W")
    m.requires("battery_mass",  unit="kg")

    m.constraint("battery_mass", lambda f:
        (f["extra_power"] + 10.0 * (9.81 * (f["battery_mass"]
                                            + f["extra_payload"])) ** 2)
        * f["endurance"] / 1.8e6
    )
    m.loop_on("battery_mass")

drone = m.build()
```

Calling `solve(drone, {"endurance": 300.0, ...})` runs the same Kleene iteration the worked example 1 uses, returning the same `battery_mass = 0.49 kg` fixed point.

12 The System Builder

For larger designs the `System` class in `codesign.system` provides modular composition with named subsystems and algebraic connection constraints. This is the recommended way to build non-trivial co-design models. Two equivalent surface syntaxes are available: the operator-overloaded form (recommended for new code) and the legacy lambda form (still supported).

12.1 Operator-overloaded syntax (recommended)

`provides`, `requires`, and `add` each return a port handle. Arithmetic operators on handles build expression trees lazily; the `>=` operator registers a constraint with the parent system.

```
from codesign import Module, Reals, System, solve

class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}
    def h(self, f):
        return {"mass": f["capacity"] / 1.8e6}

class Actuator(Module):
    F = {"lift_force": Reals(unit="N")}
    R = {"power":      Reals(unit="W")}
    def h(self, f):
        return {"power": 10.0 * f["lift_force"] ** 2}

sys = System("drone")
endurance     = sys.provides("endurance",     unit="s")
extra_payload = sys.provides("extra_payload", unit="kg")
extra_power   = sys.provides("extra_power",   unit="W")
total_mass    = sys.requires("total_mass",    unit="kg")

battery  = sys.add("battery",  Battery())
actuator = sys.add("actuator", Actuator())

battery.capacity    ≥ (actuator.power + extra_power) * endurance
actuator.lift_force ≥ 9.81 * (battery.mass + extra_payload)
total_mass          ≥ battery.mass + extra_payload

drone = sys.build()
```

The four port kinds are:

- **Outer F** (from `provides`): can appear on the RHS of constraint expressions. Cannot be a constraint target.
- **Outer R** (from `requires`): can be a constraint target. Cannot appear in expression RHS.
- **Module F** (`handle.f_port`): can be a constraint target. Cannot appear in expression RHS (its value is determined by the constraints on it, not by the current iteration state).
- **Module R** (`handle.r_port`): can appear in expression RHS. Cannot be a constraint target (its value is determined by the module's own `h`, not externally).

Misusing a port (e.g. trying to put a module F port on the RHS, or constraining an outer F) raises immediately with a message naming the port and explaining the rule. F and R port names within a single subsystem must be disjoint.

The supported operators are `+`, `-`, `*`, `/`, `**`, unary `-`, along with the helper functions `codesign.sqrt`, `exp`, and `log` for expressions that need transcendental terms. For demands that cannot be expressed algebraically, the legacy lambda form remains available and can be mixed freely with the operator form (see below).

12.2 Legacy lambda syntax

```
sys.constrain("battery.capacity",
              lambda x: (x["actuator.power"] + x["extra_power"]) * x["endurance"])
sys.constrain("actuator.lift_force",
              lambda x: 9.81 * (x["battery.mass"] + x["extra_payload"]))
sys.constrain("total_mass",
              lambda x: x["battery.mass"] + x["extra_payload"])
```

The argument `x` is a dict carrying outer F values under their bare names (`x["endurance"]`) and subsystem R ports under their dotted names (`x["battery.mass"]`). Both syntaxes compile to the same internal representation; the operator form is purely sugar.

12.3 Class-based modules

The `Module` base class lets a design problem be defined declaratively:

```
class Battery(Module):
    F = {"capacity": Reals(unit="J")}
    R = {"mass":     Reals(unit="kg")}

    def __init__(self, specific_energy=1.8e6):
        self.specific_energy = specific_energy
        super().__init__()

    def h(self, f):
        return {"mass": f["capacity"] / self.specific_energy}
```

A `Module` subclass declares its F and R ports as class-level dicts and overrides `h(self, f)`. The constructor wires everything into the underlying `DesignProblem` machinery. Parameterised modules are written by overriding `__init__` and calling `super().__init__()` after setting any instance attributes. `h` may return a dict (treated as a singleton antichain), a list of dicts (multi-valued antichain), or an `Antichain` directly.

Example 12.1 (A modular drone end-to-end)**.** *The drone from worked example 7, built with the operator-overloaded constraint syntax. Each line below a `System` declaration is a textbook inequality:*

```
sys = System("drone")
endurance     = sys.provides("endurance",     unit="s")
extra_payload = sys.provides("extra_payload", unit="kg")
extra_power   = sys.provides("extra_power",   unit="W")
total_mass    = sys.requires("total_mass",    unit="kg")

b = sys.add("battery",  Battery())
a = sys.add("actuator", Actuator())

b.capacity   ≥ (a.power + extra_power) * endurance
a.lift_force ≥ 9.81 * (b.mass + extra_payload)
total_mass   ≥ b.mass + extra_payload

drone = sys.build()
r = solve(drone, {"endurance": 300.0,
                  "extra_payload": 0.5,
                  "extra_power": 5.0})
# r.antichain → Antichain[(total_mass=0.56 kg)]
```

`build()` bundles every subsystem's R into a single Kleene axis and closes the loop automatically. The constraint graph can be exported with `viz.to_dot(drone)` for documentation.

12.4 Semantics

The `build` method emits a `Loop` whose axis bundles every subsystem's resource poset:

$$\texttt{\_\_modules\_\_} = \prod_{m \in \text{modules}} R_m.$$

The inner DP performs one Kleene step:

1. Read the current bundle estimate to populate `ctx[m.r]` for every subsystem R port.
2. For each subsystem m, evaluate the constraint demands on its F ports to obtain f_m, then call $m.h(f_m)$ to obtain an antichain.
3. Take the Cartesian product across subsystems; for each combination, evaluate the outer-R constraint demands to produce a candidate point.
4. Return the resulting antichain.

The outer Kleene iteration converges over all subsystem R ports simultaneously.

12.5 Validation

`build` performs the following checks:

- every subsystem F port has at least one `constrain` target;
- every outer R has at least one `constrain` target;
- every constraint target refers to a known module F port or a declared outer R;
- module names do not clash with the reserved axis name `__modules__` and contain no `.`.

If any check fails, `build` raises `ValueError` with a message identifying the missing or invalid item.

13 Worked Examples

The package ships twelve runnable examples under `examples/` and the same twelve as executed notebooks under `notebooks/`. Each is referenced below by its filename.

13.1 Example 1: the drone (Fig. 48 of the paper)

`01_drone.py`. The canonical MCDP from the paper. A battery powers an actuator that must lift the battery plus a payload, with extra fixed payload and avionics power. The model is a single `FunctionDP` closed by `Loop` on `battery_mass`. The example sweeps five mission profiles, three feasible and two infeasible. Iteration counts range from 8 to 41 depending on how close to the feasibility boundary the case is.

13.2 Example 2: integer optimisation (Sec. VI-D)

`02_integer_optimization.py`. The problem

$$x + y \geq \lceil \sqrt{x} \rceil + \lceil \sqrt{y} \rceil + c$$

over $\mathbb{N} \times \mathbb{N}$, parametrised by c. The inner DP enumerates all valid splits of the deficit, producing a multi-point antichain at each iteration. For $c = 4$ the iteration converges in six steps to $M(4) = \{(0,7),(3,6),(4,4),(6,3),(7,0)\}$. The example notes that the paper's claim $M(1) = \{(1,0),(0,1)\}$ contains a typo; the correct answer is $\{(0,3),(3,0)\}$, which is what the solver finds.

13.3 Example 3: AUV seabed surveying (Sec. VIII)

`03_auv_seabed.py`. An autonomous underwater vehicle sweeps an area A at velocity v with sensor radius r. Cyclic constraints couple time, energy, and cost. The example enumerates a small set of (v, r) tradeoffs per iteration, producing a three-point Pareto front per scenario.

13.4 Example 4: `UncertainDP` and `ODE_DP`

`04_uncertain_and_ode.py`. Two demos of the advanced primitives. First: a battery whose specific energy is known only to lie in $[1.6, 2.0]$ MJ/kg; the lower and upper brackets give optimistic and pessimistic masses. Second: a heater whose steady-state input power is recovered from Newton's law of cooling via `ODE_DP`.

13.5 Example 5: visualising the Kleene ascent

`05_visualize_kleene.py`. Uses matplotlib to render the sequence $A_0, A_1, \ldots$ for the Sec. VI-D problem, reproducing the structure of Fig. 36 in the paper. PNGs for $c \in \{1, 4, 8\}$ are committed under `docs/images/`.

13.6 Example 6: the drone in MCDPL syntax

`06_drone_mcdpl_syntax.py`. Rebuilds the drone with the `MCDP` builder. The model reads almost identically to the `mcdp { ... }` block in Fig. 48 of the paper. Output is identical to example 1.

13.7 Example 7: the drone, modular

`07_drone_modular.py`. Rebuilds the drone with the `System` builder. Battery and actuator are defined as `Module` subclasses with their own F and R, then wired together with three `>=` constraints in the operator-overloaded syntax. Fixed-point values match example 1 exactly; iteration counts are roughly $2\times$ because the bundle now has two coupled axes (battery mass, actuator power) instead of one.

13.8 Example 8: a modular vehicle with Pareto tradeoffs

`08_vehicle_modular.py`. Three independent subsystems: a `CatalogDP` motor (seven Pareto-incomparable entries), a linear chassis, and a battery. Five `>=` constraints close the loop on (motor, chassis, battery) resources. Two of the three mission profiles yield two-point system Pareto fronts (different motor choices trading total mass against total cost); the third is infeasible because no motor in the catalog can supply enough torque once the chassis grows to support the battery.

13.9 Example 9: a robotic arm with non-trivial topology

`09_robotic_arm.py`. Five subsystems (shoulder joint, elbow joint, sensor, controller, battery) wired with eight constraints. The connection graph is genuinely non-trivial: the controller's power demand depends on both the sensor rate and the joint command rate, the shoulder must support the elbow joint's mass at the end of its arm (introducing a mechanical cycle through `elbow.mass`), and the battery is sized by the integrated power of every electrical subsystem. The Kleene iteration resolves the resulting cycles in four steps for all three test mission profiles. This example is the cleanest demonstration of where the operator-overloaded syntax pays off: each constraint line states one physical relationship, and the cyclic structure emerges from the constraint graph rather than from explicit loop machinery.

13.10 Example 10: watching the solver work

`10_solver_trace.py`. The drone of example 7 instrumented to show every observability feature of `solve`: silent mode, the end-of-solve summary line (`verbose=1`), the per-iteration progress feed (`verbose=2`), the structured trace (`trace=True`), and a custom `on_iteration` callback. The same script runs the drone with three different `max_iter` caps and a deliberately under-resourced mission, demonstrating that `status` takes the three distinct values `"converged"`, `"max_iter"`, and `"diverged"` on these inputs. The notebook version (`10_solver_trace.ipynb`) adds a delta-vs-iteration plot showing the characteristic oscillating decay of the Kleene iteration to machine precision.

13.11 Example 11: set-based deterministic uncertainty

`11_uncertain_drone.py`. The drone's battery has two internal parameters (specific energy, efficiency) declared with "more is better" directions. First a `Box` with independent ranges, then a tilted `Ellipsoid` of the same widths. The worst-case mass under the box is $+0.0158$ kg above nominal; under the ellipsoid it is $+0.0050$ kg. The discrepancy is the cost of admitting the implausible corner where both parameters are simultaneously at their worst; the ellipsoid model rejects it explicitly.

13.12 Example 12: stochastic uncertainty with a Gaussian copula

`12_stochastic_drone.py`. The same drone with two uniform marginals tied by a Gaussian copula at correlation 0.4. A single `solve` call requests all five summaries: `worst_case`, `mean`, `p95`, `cvar95`, `samples`. Over a thousand Monte Carlo samples, the five values come out as nominal < mean < p95 < CVaR95 < worst_case, the canonical ordering. The notebook adds a matplotlib histogram of the MC distribution with vertical markers at each summary.

13.13 Example 13: the microgrid flagship

`13_microgrid.py`. An off-grid cabin must supply daily energy and peak load from four subsystems: solar PV, lithium battery (with a discrete chemistry choice over LFP, NMC, LCO, NaIon), diesel generator, and structural frame. The frame's mass depends on the total supported mass *including its own*, so the Kleene iteration does real work; iteration counts of 20–25 are typical at the test mission. The example exercises every package feature in turn: catalog choice, cyclic coupling, warm-started parameter sweep over a 50-point load range (the second pass through the sweep, reusing the previous fixed point, takes roughly 10 % fewer total Kleene steps), stochastic sun hours via `Stochastic`, and the full visualisation suite.

13.14 Example 14: online elimination over a robot catalogue

`14_online_fleet.py`. A logistics service must hit a target throughput and range from a catalogue of 200 candidate robot types, each described by four features (speed, payload, unit cost, energy per km). The exhaustive solve runs 200 inner solves; the online solver from `codesign.online` prunes by an optimistic-evaluator bound. The example compares three flavours:

- Lipschitz with $L = 300$ for cost and $L = 30$ for energy evaluates roughly 190 / 200 candidates but provably recovers all Pareto-optimal types.
- Monotonicity on a derived `cost_per_capacity = unit_cost / (speed * payload)` feature evaluates only 13 / 200 candidates and still recovers every Pareto point.
- Linear-parametric with a 3-sigma confidence band evaluates roughly 35 / 200 candidates but wrongly eliminates one Pareto point on this seed, illustrating the bias-variance tradeoff.

The notebook visualises the elimination cascade in feature space with stars marking the true Pareto front.

13.15 Example 15: monoclonal antibody fed-batch co-design

`15_bioprocess.py`. A biopharmaceutical company has to deliver a monoclonal antibody at a target titer (g/L) and annual demand (kg/year). Four subsystems are coupled cyclically: a CHO cell line characterised by an effective specific productivity and an oxygen uptake rate; a media `CatalogDP` over commercial CD formulations (HyClone-CD, EX-CELL-CD-CHO, Cellvento-CHO-220, BalanCD-HIP); a bioreactor `CatalogDP` spanning single-use 200 L through stainless-steel 25,000 L formats; and a feed strategy parameterised by glucose set-point with a U-shaped batch-failure penalty.

The cycle that the Kleene iteration resolves: higher titer demand forces higher peak cell density, which inflates the metabolic burden through the feed strategy, which inflates the cell density required to deliver the same titer, which in turn drives the bioreactor and media catalogue lookups. The outer Pareto front in $(\mathrm{COGS}, \mathrm{footprint}, \mathrm{CO_2}$ per gram) contains a genuine two-point tradeoff between CHO-K1 (low licence fee, long batch, larger footprint per year) and CHO-MK (high licence fee, short batch, smaller footprint at the same annual output).

All parameters are calibrated to the 2024-2026 bioprocessing literature: specific productivity per cell line (Reinhart *et al.* 2021, Sumi *et al.* 2024), oxygen uptake rates and k_La correlations (BioProcess International 2024), bioreactor capex (Sustainability Atlas 2026, BioPlan 2025), media pricing (CHO media market report 2025), and metabolic constraints from Khattak *et al.* 2010 and Lao & Toth 1997. The example is the first biotech-upstream application in the library and is intended both as a teaching example for the framework and as a starting point for discussions with industrial process-development groups.

13.16 Example 16: online DOE for the mAb fed-batch process

`16_online_doe.py`. Process development almost never sees a choice between fundamentally different cell lines; that decision is made years before commercialisation. The day-to-day question is where to operate within a tight 4D box of process parameters around a fixed line. This example fixes CHO-K1 and the 100 kg/year mission from example 15 and sweeps over a $5 \times 5 \times 5 \times 3 = 375$-point grid of operating conditions (temperature, pH, glucose target, feed start day). Each candidate stands in for a 10 to 14-day bioreactor run costing \$20,000 to \$100,000 in materials, labour, and analytics.

The example compares two non-online baselines (a 75-run factorial DOE at the pH = 7.1 slice, and a 40-run uniform random sample at fixed seed) against the three online evaluators from `codesign.online` at a budget of 40 inner solves. At this budget the LinearParametric evaluator and the tuned Lipschitz evaluator both recover 3 of the 4 distinct Pareto classes, matching the 75-run factorial DOE at 53% of the experimental cost. The Monotonicity evaluator alone is uninformative on this problem because its lower bounds tighten only for candidates above every observation in the partial order, and the picker has no observation at the low-feature corner where the Pareto front lives. The example flags this explicitly and discusses warm-start strategies. The companion notebook visualises the LinearParametric pick trajectory in the (temperature, glucose) projection, showing the exploration phase before `min_obs` is reached and the concentration phase that follows.

The take-home message generalises beyond bioprocess: the structural prior is doing the work, the budget is the cost. A predictive prior like LinearParametric, or a hybrid Gaussian-process-with-Lipschitz- tail, propagates information across the whole grid; pure local bounds (Lipschitz with conservative L, Monotonicity) need either denser observations or a warm-start.

14 Modelling Guidelines and Pitfalls

A few patterns that come up repeatedly when authoring MCDPs.

Expose loop variables you want to inspect. When using the operator API directly, `Loop` projects its axis out of the outer R. To inspect the converged loop value, include it in the inner R under a *different name*: for example, both `battery_mass` as the loop axis and `report_mass` mirrored for outer-R visibility. The `System` builder handles this transparently.

Cap physical maxima. When a design variable has a physical ceiling ($v_{\max}$, $r_{\max}$), make h return a $\top$-valued antichain whenever the iterate exceeds it. The Kleene ascent will then converge to infeasibility rather than oscillating or diverging.

Use `FunctionDP` or `CatalogDP` for breadth. `AlgebraicDP` always returns a singleton antichain. For a genuine Pareto front, use `FunctionDP` (enumerate the tradeoffs explicitly) or `CatalogDP` (provide multiple incomparable entries).

Scalarisation is downstream. The MCDP solver returns the Pareto front. Choosing which point to ship is a separate concern: apply `minimize_cost` with a scalar objective, or inspect the front and pick by hand.

Catalog Cartesian product can blow up. When several subsystems return multi-valued antichains, the inner h of the `System` takes their Cartesian product. For typical models (one or two catalog subsystems among many algebraic ones), this is not a bottleneck; but if many subsystems are catalog-driven, the inner antichain may grow large between Min prunings.

Numerical precision in feedback loops. `Antichain.eq` compares points exactly. For contractive floating-point feedback, the iteration may oscillate by an ulp near the fixed point and consume many iterations before converging. The divergence cap and `max_iter` guard against runaway, but choosing `max_iter` generously (200 to 500) is wise for nontrivial models.

15 Limitations and Future Work

Version 0.1.0 covers the algorithmic core: posets, antichains, six primitive DP types, three composition operators, the Kleene solver, and the two builders. The following are intentional omissions, all tractable extensions:

- **MCDPL surface syntax**. The paper's `mcdp { ... }` text format is not parsed. The `MCDP` builder gives the same shape in Python.
- **Non-finitely-representable antichains**. The library works on finite, exactly-represented antichains. For relations whose Pareto front is continuous, the only support is the bracket pattern of `UncertainDP`. Approximation kernels (Sec. VII of Censi 2015) are not yet implemented.
- **Visualisation of the design graph**. Example 5 visualises the Kleene ascent over $\mathbb{N} \times \mathbb{N}$, but there is no general renderer for the operator tree or for the subsystem network of a `System`.
- **Symbolic/automatic differentiation of** h. All relations are evaluated numerically.
- **Distributed or parallel solving**. The Kleene iteration is sequential.

Contributions, particularly on these items, are welcome.

16 API Reference Summary

The public API exported by `import codesign` is summarised below.

Posets. `Reals`, `Naturals`, `Ports`, `Discrete`, abstract base `Poset`.

Antichains. `Antichain`.

Design problems. `DesignProblem` (abstract base), `AlgebraicDP`, `FunctionDP`, `CatalogDP`, `CatalogEntry`, `ConstraintDP`, `ODE_DP`, `UncertainDP`.

Composition. `Series`, `Parallel`, `Loop` (classes); `series`, `par`, `loop` (function aliases).

Primitives. `adder`, `multiplier`, `scale`, `constant`, `identity`.

Solver. `solve`, `kleene_loop`, `minimize_cost`, `SolveResult`, `TraceEntry`. Warm-start via the `start_from=` argument on `solve`.

Builders. `MCDP`, `System`.

Class-based modules. `Module` (declarative `DesignProblem` base class).

Constraint DSL. `Port`, `Expr`, `ModuleHandle` (returned by `System.add`), and the helper functions `sqrt`, `exp`, `log` for use inside constraint expressions.

Uncertainty. Set-based: `UncertaintySet` (abstract), `Box`, `Ellipsoid`, `Disk`, `Circle`. Stochastic: `Stochastic`, plus copulas `Independence` and `GaussianCopula`. Result: `UncertaintyResult`. Entry point: `solve_with_uncertainty` (the same machinery is also reachable through `solve(dp, f, uncertainty=[...])`).

Online learning. `OptimisticEvaluator` (abstract); `MonotonicityEvaluator`, `LipschitzEvaluator`, `LinearParametricEvaluator`. Result: `OnlineResult`. Entry point: `solve_online`.

Visualisation. Imported as `from codesign import viz`. Functions: `plot_antichain`, `plot_convergence`, `plot_uncertainty`, `to_dot`.

Module-level. `__version__` (the string `"0.1.0"`).

References

[1] Andrea Censi. *A Mathematical Theory of Co-Design.* arXiv:1512.08055, 2015. https://arxiv.org/abs/1512.08055

[2] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order.* Cambridge University Press, 2nd ed., 2002.

[3] Saud Alharbi, Munther A. Dahleh, and Gioele Zardini. *Compositional Online Learning for Co-Design.* arXiv:2604.22624, 2026. https://arxiv.org/abs/2604.22624

[4] The MCDPL software distribution. https://co-design.science/software/